

SYSTEM DESIGN & SECURITY REVIEW

JOB PORTAL API (PRODUCTION & FRONTEND READINESS REPORT)

1. Executive Summary

This report details a comprehensive, blunt architectural and security assessment of the Job Portal API codebase. The review was conducted using senior-level system design best practices. It checks for critical flaws, security gaps, concurrency vulnerabilities, logic redundancies, and code elements that will break in both local testing and production environments.

Our findings reveal that the system has several critical flaws. Most notably, a syntax error in the authentication middleware makes the codebase completely unrunnable in any environment. Additionally, there are severe vulnerabilities including bypassable file upload validation, loose RBAC rules allowing privilege escalation/demotion, broken duplicate-request gates, and a concurrency bug leading to lost updates in job counters.

2. Severity Distribution

- **CRITICAL SEVERITY:** Issues that crash the application, allow unrestricted remote code execution/file upload, or permit complete authentication bypass.
 - **HIGH SEVERITY:** Vulnerabilities permitting privilege escalation, user hijacking, database corruption via concurrency race conditions, or permanent cloud storage leakage.
 - **MEDIUM SEVERITY:** Lack of database indexing, missing pagination (leading to memory exhaustion/DoS), and leaky REST routes exposing draft data.
 - **LOW SEVERITY:** Lax environment variable validations, lack of graceful error logs, and minor system redundancies.
-

3. Critical Severity Findings

3.1 syntax Blocker in Authentication Middleware

File: src/middleware/auth.middleware.js (Line 28)

In the authenticate middleware, a loose shell command token "cd" has been accidentally left inside the JavaScript block. Because Node.js parses all import modules on startup, this syntax error prevents the entire server from booting up in both development and production.

```
// src/middleware/auth.middleware.js
const user = await User.findById(decoded.id);
if (!user) {cd
    throw new ApiError(401, 'User no longer exists');
}
```

IMPACT & BREAKING CONTEXT

The server fails to start. Any attempt to build, run, or deploy this code to production or staging will immediately crash during node execution.

3.2 Bypassable File Upload & Unrestricted File Type Injection

File: src/middleware/upload.middleware.js (Line 20)

The resume upload middleware uses a boolean OR (||) operator instead of an AND (&&) operator to validate file mimetype and extension. Furthermore, it trusts client-supplied headers.

```
// src/middleware/upload.middleware.js
const hasValidMime = allowedMimeTypes.includes(file.mimetype);
const hasValidExtension = allowedExtensions.some((ext) =>
    file.originalname.toLowerCase().endsWith(ext)
);
if (hasValidMime || hasValidExtension) { ... }
```

SECURITY VULNERABILITY

An attacker can upload "payload.exe" with a spoofed "Content-Type: application/pdf" header, and the backend will accept and stream it to Cloudinary. Alternatively, they can upload a malicious binary named "payload.pdf" with an "application/octet-stream" mimetype and bypass the check. This enables hosting malware or executing serverless payload runs.

4. High Severity Findings

4.1 Concurrency lost Update in Job Applications Counter

File: src/controllers/jobApplication.controller.js (Line 40)

When a job seeker applies to a job, the application count is fetched, incremented in Node.js, and saved back to the database. This is a classic concurrency "read-modify-write" anti-pattern.

```
// src/controllers/jobApplication.controller.js (Line 40)
job.applicationsCount += 1;
await job.save();
```

CONCURRENCY IMPACT

If 50 users apply for a popular job simultaneously, multiple requests will read the same count, increment it to the same value, and write it back. This causes lost updates, leading to a corrupt "applicationsCount" in the database that is significantly lower than the actual application documents.

4.2 Broken Duplicate Job Detection (Faulty Request Gate)

File: src/controllers/job.controller.js (Line 26-30)

The controller attempts to prevent duplicate job listings by checking for an existing job with the same title, posted by the same user, at the same company. However, it queries against "req.body.company" which is undefined (as the company is resolved from req.user.companyId inside the controller).

```
// src/controllers/job.controller.js
const existing = await Job.findOne({
  postedBy: req.user._id,
  title: req.body.title,
  company: req.body.company, // req.body.company is undefined here!
});
```

LOGIC FAILURE

Because the query looks for "company: undefined", it will never match an existing job (which has an active ObjectId). This allows duplicate requests to go through, resulting in identical jobs being spammed to the feed.

4.3 Loose RBAC: Unchecked Role Updates & Privilege Demotions

File: src/controllers/company.controller.js (Line 137-139)

The "addRecruiter" function assigns the role of "recruiter" to a user using its ID, without checking their current status.

```
// src/controllers/company.controller.js
userToAdd.role = 'recruiter';
userToAdd.companyId = company._id;
await userToAdd.save();
```

RBAC COMPROMISE

An employer can pass the user ID of an ADMIN or an EMPLOYER of another company. The backend will demote the Admin or hijack the Employer, changing their role to "recruiter" and reassigning their companyId. This allows cross-company hijacking and authorization corruption.

4.4 Cloudinary Storage Leakage & Cost Overhead

File: src/controllers/resume.controller.js (Line 17, Line 87)

1. Resume uploads upload files to Cloudinary BEFORE validating the document fields. If "Resume.create" fails due to a missing title or Mongoose validation, the file remains in Cloudinary as an orphan.
2. When "deleteResume" is triggered, the document is deleted from MongoDB, but the backend NEVER requests Cloudinary to delete the raw asset.

INFRASTRUCTURE LEAK

Deleted or failed resumes remain hosted on Cloudinary forever, causing infinite storage bloat, leaking orphan files, and increasing operational billing costs.

4.5 Route-Level RBAC Gaps

File: src/routes/company.route.js (Line 14-15)

Company update and recruiter addition endpoints do not apply route guards (e.g. authorize("employer")). They rely purely on internal controller checks. While ownership is checked, it makes auditing difficult and opens the door for bugs. Furthermore, since they do not account for Admins, Admins cannot update companies or fix corrupt listings.

5. Medium Severity Findings

5.1 Missing Pagination on High-Volume Get Endpoints (DoS Vulnerability)

Endpoints: GET /api/jobs, GET /api/admin/companies/pending, GET /api/applications/job/:jobId

These endpoints execute ".find()" queries without "limit" or "skip" pagination constraints. As the database grows to thousands of records, fetching and serializing all documents in a single request will lead to severe latency, memory exhaustion, and server crashes.

5.2 Data Leakage: Draft & Closed Jobs Accessible by ID

File: src/controllers/job.controller.js (Line 78-84)

The single job lookup endpoint "getJob" queries "Job.findById(req.params.id)" and returns the job directly. Unlike the search list, it does not check if the job is a "draft" or "closed", allowing any applicant to read unpublished jobs if they guess the ID.

5.3 Unindexed Database Queries causing Full Table Scans

DB Performance Gaps:

- **Company Collection:** "isVerified" and "createdBy" fields have no indexes. Sorting or filtering pending companies triggers full table scans.
- **User Collection:** "companyId" has no index. Fetching users belonging to a company is extremely slow.
- **Interview Collection:** "company" is not indexed, slowing down the employer's interview dashboard.
- **Job Search Regex Query:** GET /api/jobs uses "new RegExp(location, 'i')". Regex searches cannot use standard indexes, resulting in slow scans.

5.4 Missing Global Exception Listeners

File: src/server.js

The server lacks listeners for "unhandledRejection" and "uncaughtException". Any asynchronous error thrown outside of the Express request cycle (e.g., database connection drops, background timers, unhandled promises) will crash the Node process instantly.

6. Error Handling & Message Gaps

6.1 Silent 500 Errors in Production

File: `src/middleware/error.middleware.js`

When an internal server error (500) occurs, the middleware catches it and sends a clean response, but it NEVER logs the error to the console (e.g., `console.error(err)`). Since stack traces are omitted in production, developers will have zero logs to debug production failures.

6.2 "process.exit" in Database Utility Breaks Tests

File: `src/config/db.js`

If the initial database connection fails, "connectDatabase" calls "process.exit(1)". In a test runner environment (Vitest), this abruptly kills the test runner process, preventing proper test suite teardown and output reporting.

7. Testing Strategy

To address these issues and verify their fixes, a comprehensive test suite using Vitest has been written under the "tests/" directory. The test suite includes:

- **Authentication & RBAC Tests:** Verifies JWT token validation, checks that unauthorized roles are blocked from restricted routes, and tests role constraints.
- **Concurrency & Duplicate Gates:** Performs concurrent requests to job applications to verify that "applicationsCount" is updated atomically, and tests the duplicate job posting gate.
- **Upload Validation Security Tests:** Verifies that files with invalid mimetypes or incorrect extensions are rejected by the upload parser.
- **Database Connection Failures:** Ensures database connection errors are thrown as catchable exceptions rather than killing the runtime.

End of Review Report
